

Discovering Predictable Latent Factors for Time Series Forecasting

Jingyi Hou , Zhen Dong , Jiayu Zhou , *Member, IEEE*, and Zhijie Liu , *Member, IEEE*

Abstract—Modern temporal modeling methods, such as Transformer and its variants, have demonstrated remarkable capabilities in handling sequential data from specific domains like language and vision. Though achieving high performance with large-scale data, they often have redundant or unexplainable structures. When encountering some real-world datasets with limited observable variables that can be affected by many unknown factors, these methods may struggle to identify meaningful patterns and dependencies inherent in data, and thus, the modeling becomes unstable and unpredictable. To tackle this critical issue, in this article, we develop a novel algorithmic framework for inferring latent factors implied by the observed temporal data. The inferred factors are used to form multiple predictable and independent signal components that enable not only the reconstruction of future time series for accurate prediction but also sparse relation reasoning for long-term efficiency. To achieve this, we introduce three characteristics, i.e., predictability, sufficiency, and identifiability, and model these characteristics of latent factors via powerful deep latent dynamics models to infer the predictable signal components. Empirical results on multiple real datasets show the efficiency of our method for different kinds of time series forecasting tasks. Statistical analyses validate the predictability and interpretability of the learned latent factors.

Index Terms—Financial time series, latent factor model, sequence modeling.

I. INTRODUCTION

TIME series forecasting is an essential task of data science and machine learning, with broad applications spanning areas like financial management, health informatics, weather forecasting, and epidemic prediction. Deep learning has shown

its strong ability to model various complex sequential information in many fields, especially for vision and language tasks [1], [2], [3]. Even though there are many efforts applying deep learning in time series forecasting, unlike the data in successful deep learning applications, most real-world time series neither contain sufficient information with high-dimensional data nor represent explicit semantic information that can guide the data analysis. Therefore, deep learning methods still face critical challenges in time-series forecasting tasks.

A particular challenge lies in the inherent spatio-temporal complexity of time series data, as the underlying patterns can be influenced by a multitude of unknown factors. Discovering and modeling these factors and their relations empower time series forecasting with good interpretability and thus greatly enhance the forecasting performance of models. Prior work has delved into the use of factors and their relations to forecast time series, either by explicitly exploiting additional knowledge [4], [5], [6], [7] or explicitly modeling relations of latent representations [8]. These approaches may have limited applicability and are often feasible for specific tasks with the need for a sophisticated process of domain knowledge. However, real-world time series could be potentially affected by various kinds of factors from different domains, and it is infeasible to explicitly collect a comprehensive set of all the factors. For example, in the financial domain, asset volatility describes the price changes of an asset (e.g., a stock), and its prediction is important in many quantitative strategies. Asset volatility is affected by massive factors that are hard to observe or measure, e.g., macro economy, investor sentiment, and various events in the markets. Moreover, real-world relations between different factors over time are highly complex, with some factors being unobservable, which further complicates precise modeling.

Fortunately, since some observable data are dependent on these unobserved factors, it is possible to learn a graphical model that can infer latent factors from observations generatively. This allows us to learn and disentangle latent factors with characteristics that support rational relation modeling to explain the generation procedure of time series, which benefits further forecasting. Specifically, the inferred latent factors are used to form various simple and intrinsic temporal signals that are easy to forecast, where “intrinsic” denotes factors in the same signal component are self-descriptive and not affected by factors from other signal components. Through this straightforward inductive bias, we can make reasonable independence assumptions on these latent factors to encourage sparsity in their relations, reducing computational complexity. As the derived temporal

Manuscript received 14 March 2023; revised 4 October 2023; accepted 11 November 2023. Date of publication 21 November 2023; date of current version 4 October 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2020YFC1523200 and in part by the Natural Science Foundation of China under Grants 62106021 and U20A20225. Recommended for acceptance by M. A. Cheema. (*Corresponding author: Jingyi Hou.*)

Jingyi Hou and Zhijie Liu are with the School of Intelligence Science and Technology, University of Science and Technology Beijing, Beijing 100083, China, also with the Institute of Artificial Intelligence, University of Science and Technology Beijing, Beijing 100083, China, and also with the Key Laboratory of Intelligent Bionic Unmanned Systems, Ministry of Education, University of Science and Technology Beijing, Beijing 100083, China (e-mail: houjingyi@ustb.edu.cn; liuzhijie@ustb.edu.cn).

Zhen Dong is with the College of Mathematics and Computer Science, Yan'an University, Yan'an 716000, China (e-mail: cndongzhen@gmail.com).

Jiayu Zhou is with the Department of Computer Science and Engineering, Michigan State University, East Lansing, MI 48823 USA (e-mail: jiayuz@msu.edu).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TKDE.2023.3335240>, provided by the authors.

Digital Object Identifier 10.1109/TKDE.2023.3335240

signals are simple and intrinsic, it is easy to forecast the future factors of each signal, and therefore, predict future data given the predicted factors.

Numerous efforts have been dedicated to decomposing temporal signals into sub-components that are more steady or regular than the original signal. Recent advancements in deep time series forecasting methodologies [9], [10], [11] employ seasonal-trend decomposition [12], which partitions time series into interpretable and reliable components, including trend, seasonal, and residual elements. However, decomposition based on predefined rules might fail in the scenario where time series are affected by various factors and barely show explicit regularities. For example, Zeng et al. [13] show that for financial time series forecasting, state-of-the-art methods demonstrate sub-optimal performance compared to a basic approach that simply relies on repeating the last observed value within the look-back window. The non-stationary signal decomposition techniques, including empirical mode decomposition (EMD) [14] and its variations [15], [16], effectively decompose time series into stationary intrinsic mode functions (IMFs) that dynamically represent the local oscillatory patterns at different time scales. Neural basis expansion analysis approaches [10], [17] hierarchically decompose the input signal into easily reconstructable sub-signals with the doubly residual mechanism. Regardless of whether deep learning techniques are employed, these empirical decomposition methods focus on the characteristics of the original time series, while our method pays attention to drawing patterns from the observations in statistics. Beyond signal decomposition, we endeavor to automatically infer easily predictable factors from complex time series data by modeling probabilistic relations between these factors and observations. In addition, we assume pairwise independence between factors of different derived temporal signals to ensure the signals' intrinsicity and sparse relations for efficient computation.

Accordingly, we aim to discover latent factors to form multiple signal components that are simple and intrinsic for prediction (i.e., predictability), informative for reconstruction (i.e., sufficiency), and feasible for learning (i.e., identifiability). To achieve this, we propose a novel approach for inferring predictable factors for forecasting time series. The proposed framework is constructed based on variational inference and sequence models by approximating the joint distribution over observed time series and latent factors following the aforementioned key characteristics of predictability, sufficiency, and identifiability. The framework can also be decomposed into multiple latent dynamics models that are conditionally pairwise independent for alleviating the error accumulating of the long-term series. We conduct empirical studies on two typical time series forecasting tasks, i.e., long-term series forecasting and stock trend forecasting, to show the effectiveness of our method, and we also conduct statistical analyses to gain insights into the predictability and interpretability of the inferred signal components.

The contributions of this work are summarized as follows:

- We consider inferring latent factors varying along time from the observation to form multiple signal components with characteristics of predictability, sufficiency, and identifiability. The factors we inferred can be used to easily construct future signals for time series forecasting.

- We design a novel method that disentangles the latent factors and models the relations of the factors according to the signal components to draw the joint distribution for more efficient approximation.
- The proposed method achieves better performance compared with the state-of-the-art for long-term series forecasting and stock trend forecasting, respectively.

II. RELATED WORK

A. Time Series Forecasting

Time series forecasting is challenging and has long been investigated in many research areas. Several classical methods are proposed for industries, such as the autoregressive methods [18], [19], moving average methods [20], filtering models [21], [22], and factor models [23], [24]. Although these methods are limited in their ability to handle data available nowadays that become increasingly large-scale and high-dimensional, they still provide theoretical guidance for time series forecasting methods.

With the notable rise of deep learning, sequence-based neural networks are getting increasingly involved. One typical structure is the recurrent neural network (RNN) which is able to process theoretically arbitrary long-term input sequences with only a few parameters [25], [26]. Despite these advantages, the recursive mechanism is not really conducive to information propagation. Attention-based RNNs [27], [28], [29] address this problem by encoding long-term series with adaptive weighted summation to generate predictions. Another structure is the convolutional network that captures various local temporal patterns with different convolutional filters [30], [31], [32]. Recently, Transformer-based models [11], [29], [33], [34], [35], [36] have emerged, treating time series as undirected graphs with nodes as time steps and edge weights as attention between nodes. Although Transformers have strong model-fitting abilities, they still suffer from high computation complexity. To reduce parameters, RNNs, convolutional networks, and Transformers make some assumptions (e.g., sparsity and locality) at the cost of impairing the long-term modeling capability. Differently, our method focuses on inferring easy-to-predict factors from the original time series, so that we can use these relatively simple predictors (without strong fitting ability) to process the factors and then calculate the future data according to the predicted factors.

The structures of our methods can be regarded as deep latent dynamics models [37], [38], [39] by considering the original time series as observations and the prediction of each factorized signal component as the decision-making process. This helps mitigate accumulating errors and improve the accuracy of long-term predictability estimation. There are methods [40], [41] studying the non-linear independent component analysis (ICA) to disentangle latent factors of dynamic models. The disentanglement can be effectively learned by these models thanks to their theoretical guarantees of the models' identifiability. Nevertheless, these methods mainly focus on designing generalized models without being constrained by too many restrictive assumptions. Differently, our method focuses on using the disentanglement of the factors to model reasonable and sparse relations for more efficient approximation.

Some methods use GNNs [42], [43], [44], [45] or MRFs [46] to model spatiotemporal relations between different time series variables to improve the forecasting performance. However, reasoning and propagating information through global relations could incur computational costs and introduce redundancy as data scale and dimension increase. We propose to use variational inference to draw distributions over latent factors and learn relations between them to generate and predict time series. Duan et al. [8] leverage VAEs to learn the multi-dynamic-factor model [47] for predicting cross-sectional stock return, ignoring the temporal variation of factors. Differently, our method takes the temporal relations into account for more comprehensive modeling and adapts the classic theory with fewer constraints by incorporating deep learning techniques.

There are methods using extra knowledge of the market to improve the performance of times series forecasting, such as the event-driven approaches [4], [5], [6] and the method exploiting additional enterprise information [7] for stock trend forecasting. Although these methods utilize most information from the environment, our method has the advantage of discovering complementary factors by analyzing the data along the temporal dimension to further boost the prediction performance, as evidenced by our experiments.

B. Decomposition of Time Series

Decomposing non-stationary observed time series into simple and predictable components is quite an intuitive way for time series analysis, and can be traced back to the blind source separation [48]. Empirical mode decomposition methods [14], [15], [16] iteratively decompose time series into stationary oscillating components, known as intrinsic mode functions (IMFs). The IMFs are simple and nearly orthogonal signals and are defined to have a similar number of extrema and zero crossings as well as symmetric envelopes to guarantee the conduct of the Hilbert transform on them. Instead of the delicately predefined intrinsic modes that should be oscillatory and add up to be the original signal, with the help of deep learning techniques, our method generalizes the data processing from the frequency domain to the concept domain by preserving the simplicity and orthogonality of the intrinsic modes that are in accordance with our instincts.

Researchers have recently ventured into constructing deep networks for time series decomposition. In these endeavors, various approaches [9], [10], [11], [36] explicitly decompose time series into trend and seasonality components, subsequently utilizing these components to forecast future signals. Sen et al. [32] use temporal convolution networks to perform matrix factorization for prediction time series. Wang et al. [49] use isometric and multi-scale convolutions to capture local-global interactions within time series data. These methods mainly focus on leveraging the existing classic signal processing theories to enhance model interpretability, while our method harnesses the potent information abstraction capabilities of deep learning to extract more intrinsic representations as predictable latent factors for time series forecasting. Except for the season-trend decomposition, Transformer-based methods, i.e., Autoformer [11] and FEDformer [36], additionally

introduce new attention mechanisms as inner decomposition modules according to the frequency characteristics of signals. Neural basis expansion analysis approaches methods like N-BEATS [10] and N-HITS [17], use residual mechanisms to hierarchically decompose predictions into interpretable outputs augmenting the seasonality-trend decomposition paradigm. Unlike gradually decomposing the original signal, our method infers latent factors to form intrinsic and predictable signal components from the input data with the guidance of semantics, which is more flexible for relation modeling.

C. Multi-Scale Temporal Sampling

Since the multiple signal components are generated using a multi-scale sampling strategy in this article, we review the sequence prediction methods that also benefit from this strategy. Temporal convolutional network (TCN) [50] stacks dilated convolutions with different dilation rates that can capture multi-scale temporal information for vision and language modeling and prediction. TS2Vec [51] uses hierarchical contrastive learning to capture scale contextual information of time series. While these methods mainly focus on extracting local contextual patterns with different temporal granularities, our method uses multi-scale sampling to discover multiple latent semantics with different periodic properties hidden in time series. Methods cascade multi-scale sampling in RNNs, such as DiLRNN [52] and LSTNet [31] to further improve long-term memory capability of models. N-HITS [17] applies multi-scale sampling to the input signal to construct the prediction hierarchically. Compared to the above methods, we take full advantage of multi-scale sampling as an inductive bias to guide the learning of latent factors with the time-scale semantics and the modeling of sparse relations between latent factors.

III. PREDICTABLE LATENT FACTOR INFERENCE MODELING

We aim to discover predictable latent factors of time series data for efficient forecasting. To achieve this, we first model the relations among the latent factors and the observed time series, and then learn the latent factors from the time series based on the modeled relations. Specifically, we assume latent factors can be divided into several independent signal components, with conditional independence between factors within each component for computational efficiency. Since the inferred latent factors drive the trends of the times series, the original time series data can be analyzed in a latent semantic domain. To effectively learn latent factors for feasible and effective prediction, we enforce the factors in each signal component with the characteristics of predictability, sufficiency, and identifiability, which will be elaborated in Section IV.

Fig. 1 shows the graphical model of the proposed predictable factor learning method. Let $\mathbf{X} = [\mathbf{x}(1), \dots, \mathbf{x}(T)] \in \mathbb{R}^{D \times T}$ be the matrix of input time series of past T time steps with D representing the number of features, and $\mathbf{H} = [\mathbf{h}(1), \dots, \mathbf{h}(T)] \in \mathbb{R}^{L \times K \times T}$ are the factorized K signal components each of which is composed of L predictable latent factors. The observed time series $\mathbf{X}$, e.g., the price signal of stocks, are usually non-stationary time series that cannot be predicted, and we assume

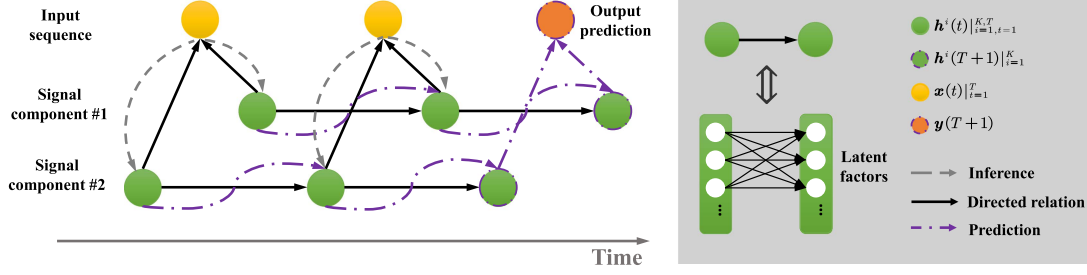


Fig. 1. Graphical model for learning predictable latent factors. We present a model with two signal components for illustration here.

that $\mathbf{X}$ depends on the hidden predictable $\mathbf{H}$ which can be inferred from $\mathbf{X}$ to accomplish time forecasting tasks. With the predicted $\mathbf{h}(T+1)$, we can make a specific prediction $\mathbf{y}(T+1)$ according to different kinds of time series forecasting.

A. Multi-Scale Convolutional Encoder

Computational efficiency is among the main challenges in time series analysis. To improve the computational efficiency, we thereby make several independent assumptions to encourage the sparsity of the relations between the inferred factors. The first independent assumption is that factors representing different time-scale semantics are merely related. Taking the financial time series data as an example, the market sentiments and the business scopes are both causal factors of the stock prices while they are independent. Market sentiments might have weekly seasonality, e.g., hesitant on Mondays and active on Wednesdays. And the profits of companies with specific business scopes usually have yearly seasonality, e.g., relatively high in summer for a company selling air-conditioners. More supportive statements are claimed by [53]. The second assumption we make is that factors in the same time scale are conditionally independent. This assumption leverages temporal information to clearly describe the directed relations between latent factors. If two factors are related to each other, the factor at the previous time step determines the current time step, and these two factors are independent given the observations at the current time step.

With the two independence assumptions above, we now discuss the use of a multi-scale convolutional encoding network, which can discover various predictable latent factors and directly divide the factors into different independent signal components according to the time scales. Concretely, we sample the observed time series data in several specific time scales corresponding to different semantics, such as “day” and “week”. Let $\mathcal{R} = \{r_1, \dots, r_K\}$ be the sets of K sampling rates, the sampled time series are $s(\mathbf{X}, \mathcal{R}) = \{s(\mathbf{X}, r_i) | i = 1, \dots, K\}$, where $s(\mathbf{X}, r)$ describes the sampling operation on $\mathbf{X}$ with sampling rate of r . The proposed encoder ϕ learns latent factors at each time scale individually, and the factors are combined during the decoding procedure. In particular, the ϕ is composed of K sub-encoders $\phi = \{\phi^{(i)}\}_{i=1}^K$, and these sub-encoders are used to learn the distributions of latent factors:

$$\begin{aligned} [\boldsymbol{\mu}^i, \boldsymbol{\sigma}^i] &= \phi^{(i)}(\mathbf{X}^{(i)}), \mathbf{X}^{(i)} = s(\mathbf{X}, r_i), \\ \mathbf{h}_j^i(t) &\sim \mathcal{N}(\boldsymbol{\mu}_j^i(t), \boldsymbol{\sigma}_j^i(t)), j = 1, 2, \dots, L, \end{aligned} \quad (1)$$

where $\mathbf{h}_j^i(t)$ represents the j -th latent factor learned by the input time series with a sampling rate of r_i (i.e., the j -th latent factor in the i -th signal component) at time step t . We use a dilation convolution operation to directly implement the function composition $\phi^{(i)} \circ s$ with appropriate padding to hold the same size between $\mathbf{X}^{(i)}$ and $\mathbf{X}$. The $\boldsymbol{\mu}_j^i(t)$ and $\boldsymbol{\sigma}_j^i(t)$ represent the j -th element of $\boldsymbol{\mu}^i \in \mathbb{R}^{L \times T}$ and $\boldsymbol{\sigma}^i \in \mathbb{R}^{L \times T}$ at time step t , respectively. So $\mathcal{N}(\boldsymbol{\mu}_j^i(t), \boldsymbol{\sigma}_j^i(t))$ in (1) shows that the learned posterior distribution of the latent factor is a 1-d Gaussian distribution with a mean value of $\boldsymbol{\mu}_j^i(t)$ and standard deviation of $\boldsymbol{\sigma}_j^i(t)$.

B. Co-Attention-Weighted Decoder

After encoding the historical observations into K signal components, we present a co-attention-weighted decoder that includes two sets of sub-decoders with the same weight coefficients for interpolating the sub-decoders of each set. The two sets aim to reconstruct the observation $\mathbf{X}$ and the prediction $\mathbf{Y}$, respectively. The weights are set to be shared because both of them represent the importance of the signal components contributing to the formation of the observed data.

First, a linear transformation $f_d^{(i)}$ is used to reconstruct $\hat{\mathbf{X}}^{(i)}$ for each time scale separately, i.e., $\hat{\mathbf{X}}^{(i)} = f_d^{(i)}(\mathbf{h}^i)$. We calculate the weights of the sub-decoders according to the reconstruction of the original historical data rather than the predictions to avoid unnecessary error accumulation. An attention mechanism is used to combine the reconstructed signals at different time scales, which is formulated as:

$$\begin{aligned} \alpha_i &= \frac{\exp(-\|\mathbf{X} - \hat{\mathbf{X}}^{(i)}\|_F^{1/2})}{\sum_{i=1}^K \exp(-\|\mathbf{X} - \hat{\mathbf{X}}^{(i)}\|_F^{1/2})}, \\ \hat{\mathbf{X}} &= \sum_{i=1}^K \alpha_i \hat{\mathbf{X}}^{(i)}, \end{aligned} \quad (2)$$

where α_i is the attention-based weight coefficient, and $\|\cdot\|_F$ is the Frobenius norm of a matrix. Gathering the linear transformation in each signal component, the decoder set for reconstructing the observation is described as $\psi_d = \{f_d^{(i)}\}_{i=1}^K$, and we further have $\hat{\mathbf{X}} = \psi_d(\mathbf{H})$.

Second, for each signal component, an RNN-based sequence model $g^{(i)}$ is introduced to derive the next hidden state $\hat{\mathbf{h}}^i(t+1) = g^{(i)}(\mathbf{h}^i(j)|_{j=t-\epsilon+1}^t)$ given the historical time series before time step $t+1$, where $\epsilon \in \mathbb{N}^+$ denotes the length of time steps

needed to predict the former latent factors. By using $\hat{\mathbf{h}}^i(t+1)$ and the shared weight coefficient α_i , the final task-specified prediction can be calculated as

$$\hat{\mathbf{Y}}(t+1) = \sum_{i=1}^K \alpha_i f_y^{(i)}(\hat{\mathbf{h}}^i(t+1)), \quad (3)$$

where $f_y^{(i)}$ is designed as an arbitrary neural network for the downstream prediction task. The decoder set for reconstructing the final prediction is thus formulated as $\psi_f = \{f_y^{(i)}\}_{i=1}^K$, and we further have $\hat{\mathbf{Y}} = \psi_f(\hat{\mathbf{H}})$.

IV. MODEL LEARNING

The proposed model encodes $\mathbf{X}$ into probability distributions of predictable latent factors $\mathbf{H}$, and $\mathbf{H}$ is used to generate the predictions of downstream tasks. The encoded factors are encouraged to have the following three characteristics:

- **Predictability.** As our presumption, there exist predictable latent factors underneath the unpredictable non-stationary time series.
- **Sufficiency.** The sufficiency means the encoded factors contain enough information for the reconstruction and prediction.
- **Identifiability.** An identifiable model guarantees the correctness of drawing the joint distribution of the observations and latent signals, where the latent factors determine the trend of the observed time series.

A. Learning Objective

To achieve this goal, we propose a learning strategy that jointly maximizes the marginal probability of the proposed graphical model via variational inference and minimizes the task-specific objective. Beforehand, an extra observed variable $\mathbf{E}$ is introduced to hold the identifiability of the whole model and disentangle the learned latent factors. We define $\mathbf{E} = [e_1, \dots, e_T] \in \mathbb{R}^{(D+1) \times T}$ with e_t describing the joint representation of the time step t and the corresponding observed signal $\mathbf{x}_t$. With the help of $\mathbf{E}$, the prior distribution $p(\mathbf{H}|\mathbf{E})$ is assumed to be conditionally factorial, where all the latent factors $\mathbf{h}_j$ are independent each other. In our model, all of the $\{\phi^{(i)}\}_{i=1}^K$ jointly model the learned posterior distribution $q_\phi(\mathbf{H}|\mathbf{X}, \mathbf{E})$ with ϕ as a parameter, which is used as a variation approximation to the true posterior distribution $p_\psi(\mathbf{H}|\mathbf{X}, \mathbf{E})$ calculated by taking a marginal distribution of the joint probability $p_\psi(\mathbf{H}, \mathbf{X}|\mathbf{E})$. ψ represents the total parameters of the decoder ψ_d and the parameters of prior distributions ψ_p , so the joint distribution is described as:

$$p_\psi(\mathbf{H}, \mathbf{X}|\mathbf{E}) = p_{\psi_d}(\mathbf{X}|\mathbf{H})p_{\psi_p}(\mathbf{H}|\mathbf{E}). \quad (4)$$

Our model optimizes the parameters by maximizing the log-likelihood function:

$$\max_{(\mathbf{X}, \mathbf{Y}, \mathbf{E}) \in \mathcal{D}} \log p(\mathbf{X}|\mathbf{E}) + \lambda \log p(\mathbf{Y}|\mathbf{X}), \quad (5)$$

where $\mathcal{D}$ is the training set, and $\mathbf{Y}$ varies with the prediction task, i.e., the future observed data in the long-term series forecasting task, and the price change rate in the stock trend forecasting task. The two terms in (5) focus on different aspects: $\log p(\mathbf{X}|\mathbf{E})$ aims to reconstruct $\mathbf{X}$ so that an informative representation $\mathbf{H}$ can be learned, and $\log p(\mathbf{Y}|\mathbf{X})$ serves for predications to encourage $\mathbf{H}$ to be predictable and discriminative, and λ is a hyper-parameter to tune the relative importance of the two terms. In this way, our method gains the advantages of both the generative and discriminative models to learn good representations for the specific forecasting task.

(5) is equivalent to

$$\begin{aligned} \max_{\phi, \psi, v, \psi_f} \sum_{(\mathbf{X}, \mathbf{Y}, \mathbf{E}) \in \mathcal{D}} \mathbb{E}_{q_\phi(\mathbf{H}|\mathbf{X}, \mathbf{E})} [\lambda \log p_{v, \psi_f}(\mathbf{Y}|\mathbf{H}) \\ + \log p_\psi(\mathbf{H}, \mathbf{X}|\mathbf{E}) - \log q_\phi(\mathbf{H}|\mathbf{X}, \mathbf{E})], \end{aligned} \quad (6)$$

where $v = \{g^{(i)}\}_{i=1}^K$ denotes the total parameters of the predictive sequence models for hidden states, and v together with ψ_f form the task-specific prediction networks. The deduction from (5) to (6) is elaborated in the Appendix, available online.

Under the assumption of conditionally factorial, we show that (6) can further be expanded as

$$\begin{aligned} \max_{\phi, \psi, v, \psi_f} \sum_{(\mathbf{X}, \mathbf{E}) \in \mathcal{D}} [\lambda \mathbb{E}_{q_\phi(\mathbf{H}|\mathbf{X}, \mathbf{E})} \log p_{v, \psi_f}(\mathbf{Y}|\mathbf{H}) \\ + \sum_{i=1}^K \mathbb{E}_{q_{\phi^{(i)}}(\mathbf{h}^i|\mathbf{X}^{(i)}, \mathbf{E})} (\log p_{\psi^{(i)}}(\mathbf{h}^i, \mathbf{X}^{(i)}|\mathbf{E}) \\ - \log q_{\phi^{(i)}}(\mathbf{h}^i|\mathbf{X}^{(i)}, \mathbf{E}))]. \end{aligned} \quad (7)$$

We provide details of the transformation procedure of (6) and (7) in the Appendix, available online. In the following, we give further explanation of (7) with respect to the three characteristics.

B. Predictability

Given a 1-D sequence $\mathbf{s} = \{s(1), s(2), \dots, s(t), \dots\}$ where $t \in \mathbb{N}^+$ is the time step and $s(t) \in \mathbb{R}$, the predictability of $\mathbf{s}$ is intuitively defined as

Definition 1: For any $\tau > 0$, there exist $\epsilon, \delta \in \mathbb{N}^+$ and a function $g: \mathbb{R}^\epsilon \rightarrow \mathbb{R}$ which satisfies that the set $\{\mathbf{s} \in \mathbb{R}^\epsilon | \lim_{\mathbf{s} \rightarrow \mathbf{s}'} g(\mathbf{s}) \neq g(\mathbf{s}')\}$ has measure zero, and for any $t > \epsilon$,

$$|g(\mathbf{s}(i))_{i=t-\epsilon+1}^t - \mathbf{s}(t+\delta)| < \tau, \quad (8)$$

where τ is a small error tolerance when the sequence $\mathbf{s}$ is predictable.

Eq. (8) suggests that historical data of a time series can be transformed into future data through continuous mapping, which is independent of time steps. If such a mapping exists, it indicates predictability within the time series data. This mapping is continuous to ensure that predicted values remain stable and do not undergo significant changes due to minor fluctuations in historical data. The mapping is independent of time steps, implying that the time series data itself exhibits specific patterns. According to the definition, the function g in

(8) is essential to the predictability of $\mathbf{x}$, but it is often difficult to find or design. As described in (3), we propose to use a sequential neural network to approximate g for endowing the latent factors with predictability. For each signal component, an RNN-based model $g^{(i)} (i = 1, 2, \dots, K)$ is introduced and $|\hat{\mathbf{h}}^i(t+1) - \mathbf{h}^i(t+1)| < \tau$ is expected where $\hat{\mathbf{h}}^i(t+1)$ is the output of the model. In general supervised learning, $\mathbf{h}^i(t+1)$ is treated as the ground-truth for training $g^{(i)}$. However, $\mathbf{h}^i(t+1)$ is calculated by the encoder $\phi^{(i)}$, and it is undetermined before the end-to-end training, which might degenerates the leaning of g . Instead, both of $\mathbf{h}^i(t+1)$ learned by $\phi^{(i)}$ and $\hat{\mathbf{h}}^i(t+1)$ predicted by $g^{(i)}$ are used to reconstruct the original historical and future signals $\mathbf{X}$, respectively, that are determinate and common. We show that feasibility is warranted in the following result.

Theorem 1: For any $\tau_1, \tau_2 > 0$, there exist $\epsilon, \delta \in \mathbb{N}^+$, a function $g: \mathbb{R}^\epsilon \rightarrow \mathbb{R}$ which satisfies that the set $\{\mathbf{s} \in \mathbb{R}^\epsilon | \lim_{\mathbf{s} \rightarrow \mathbf{s}'} g(\mathbf{s}) \neq g(\mathbf{s}')\}$ has measure zero, and a continuous and derivable function $l: \mathbb{R} \rightarrow \mathbb{R}$, for any $t > \epsilon$,

$$|l(g(\mathbf{s}(i)|_{i=t-\epsilon+1}^t))| < \tau_1, |l(\mathbf{s}(t+\delta))| < \tau_2 \quad (9)$$

holds, then $\mathbf{s}$ must be predictable.

Proof. We rewrite $g(\mathbf{s}(i)|_{i=t-\epsilon+1}^t)$ as $\hat{\mathbf{s}}$ for simplicity. Without loss of generality, $\hat{\mathbf{s}}$ is assumed to be less than $\mathbf{s}(t+\delta)$. Since that l is continuous and derivable, there must exist $\hat{\xi} < \xi < \mathbf{s}(t+\delta)$ such that

$$l'(\xi)(\mathbf{s}(t+\delta) - \hat{\mathbf{s}}) = l(\mathbf{s}(t+\delta)) - l(\hat{\mathbf{s}}), \quad (10)$$

according to the Lagrange mean value theorem. For any $\tau > 0$, let $\tau_1 = \tau_2 = \tau \times |l'(\xi)|/2$, then we have $|l(\hat{\mathbf{s}})| < \tau_1$ and $|l(\mathbf{s}(t+\delta))| < \tau_2$. Accordingly,

$$\begin{aligned} |l(\hat{\mathbf{s}}) - l(\mathbf{s}(t+\delta))| &\leq |l(\hat{\mathbf{s}})| + |l(\mathbf{s}(t+\delta))| \\ &< \tau_1 + \tau_2 = \tau \times |l'(\xi)|. \end{aligned} \quad (11)$$

Substituting (10) into (11), we thus have that

$$|l'(\xi)|(\mathbf{s}(t+\delta) - \hat{\mathbf{s}}) < \tau \times |l'(\xi)|, \quad (12)$$

and further $|g(\mathbf{s}(i)|_{i=t-\epsilon+1}^t) - \mathbf{s}(t+\delta)| < \tau$. Therefore, $\mathbf{s}$ is predictable. $\square$

C. Sufficiency

The learned components should be informative enough for both the forecasting task and reconstructing $\mathbf{X}$, and it is implemented by learning the proposed co-attention-weighted decoder that should be designed according to specific time series forecasting tasks. In this article, we present two typical tasks as examples to show the prediction learning ability of our model, i.e., long-term series forecasting and stock trend forecasting.

Long-term series forecasting: The goal of this task is to predict the future H -horizon data $\mathbf{Y} = [\mathbf{x}(T+1), \mathbf{x}(T+2), \dots, \mathbf{x}(T+H)] \in \mathbb{R}^{D \times H}$ given historical data before time step $T+1$. We apply the non-dynamic decoder to our method for generating $\mathbf{Y}$. To be specific, an MLP is used as $f_y^{(i)}$ to calculate all the H -horizon prediction of the i -th sampled time

series given the predicted hidden state $\hat{\mathbf{h}}^i(T+1)$. We use the mean squared error (MSE) loss to guide the learning of this characteristic with respect to the reconstruction of future data (i.e., forecasting). In this task, the output prediction is just the future data of the original time series, which is known as the auto-regressive generation task, so this objective is equivalent to learning the characteristic of predictability.

Stock trend forecasting: Different from the aforementioned auto-regressive generation task, stock trend forecasting aims to predict the stock price change rate given some stock-related variables (e.g., the stock price at specific times and the trading volume). The prediction $\mathbf{Y}$ is defined as $\mathbf{Y}(T+1) = (P(T+1) - P(T))/P(T)$ where P is the real price. It is obvious that $\mathbf{Y}$ is defined in two consecutive time steps, but sampling for multiple signal components would break the consecutiveness of $\mathbf{h}^i$. Fortunately, the designed function g ensures $\mathbf{h}^i$ holding the predictability, so the predicted $\hat{\mathbf{h}}^{(i)}$ is just capable of filling the time gaps. Suppose that the latent components $\mathbf{h}^i$ learned by using the sampled $\mathbf{X}^{(i)}$ are represented as $\{\dots, \mathbf{h}^i(t_1), \mathbf{h}^i(t_2), \dots, \mathbf{h}^i(T)\}$ where $t_k - t_{k-1} = \eta$ is a constant, then we use $g^{(i)}$ to predict $\{\dots, \hat{\mathbf{h}}^{(i)}(t_1+1), \hat{\mathbf{h}}^{(i)}(t_2+1), \dots, \hat{\mathbf{h}}^{(i)}(T+1)\}$, and combine them together to learn representations for the forecasting task, i.e.,

$$\begin{aligned} \{\dots, (\mathbf{h}^i(t_1), \hat{\mathbf{h}}^{(i)}(t_1+1)), (\mathbf{h}^i(t_2), \hat{\mathbf{h}}^{(i)}(t_2+1)), \dots, \\ (\mathbf{h}^i(T), \hat{\mathbf{h}}^{(i)}(T+1))\}. \end{aligned} \quad (13)$$

Using representations of (13) as inputs, another RNN-based model is used as $f_y^{(i)}$ to predict $\mathbf{Y}$. As shown in (3), the α_i in (2) is used to integrate the outputs of sequence models in various signal components. An MSE regression loss between the predicted price return and the ground-truth is calculated as the guidance of factor learning for keeping the sufficiency of information.

Furthermore, for both tasks, the sub-decoders composed of linear functions for reconstruction are designed as simple as possible to enforce $\mathbf{H}$ informative enough for reconstructing $\mathbf{X}$. However, the predictability and identifiability require an injective decoder which is often approximated by a complex neural network. The contradiction is alleviated by introducing the sum-injective theorem [54] as shown in Section IV-D.

D. Identifiability

The identifiability of our model is ensured by Theorem 1 of [40]. In order to fit the theorem, the prior distribution should be elaborated. Different from the standard normal distribution prior used in the classical VAE, the Gaussian location-scale family is chosen as the prior distribution $p_{\psi_p}(\mathbf{h}_j^{(i)} | \mathbf{E})$, which is rewritten in the form of exponential family distribution as

$$\frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{\mu^2}{2\sigma^2}\right) \exp\left(\left[\mathbf{h}_j^i \quad (\mathbf{h}_j^i)^2\right]^\top \left[\begin{array}{c} \frac{\mu}{\sigma^2} \\ -\frac{1}{2\sigma^2} \end{array}\right]\right), \quad (14)$$

where $\mathcal{T}^{(i)}(\mathbf{h}_j^i) = (\mathbf{h}_j^i, (\mathbf{h}_j^i)^2)$ are sufficient statistics, and the corresponding parameters $\lambda^{(i)}(\mathbf{E}) = (\frac{\mu}{\sigma^2}, -\frac{1}{2\sigma^2})$ and the normalizing constant $\frac{1}{\sqrt{2\pi\sigma}} \exp(-\frac{\mu^2}{2\sigma^2})$ are dependent on $\mathbf{E}$. In our model, the λ^i are independently and randomly sampled according to $\mathbf{E}$ to achieve the aforementioned characteristic of conditionally independent of the latent factors, and $\mathcal{T}^{(i)}$ are same for different components. Keeping consistent with [40], the parameters of the joint distribution are $\psi = \{\psi_d, \psi_p\}$ where $\psi_d = \{f_c^{(i)}\}$ and $\psi_p = \{\mathcal{T}^{(i)}, \lambda^{(i)}\}$ when gathering the parameters of all components. For the deep latent variable model, the $\sim_A$ identifiable is introduced as

Definition 2: Define $\sim$ as the equivalence relation:

$$(f, \mathcal{T}, \lambda) \sim (\tilde{f}, \tilde{\mathcal{T}}, \tilde{\lambda}) \Leftrightarrow \exists \mathbf{A}, \mathbf{c} | \mathcal{T}(f^{-1}(\mathbf{x})) = \mathbf{A}\tilde{\mathcal{T}}(\tilde{f}^{-1}(\mathbf{x})) + \mathbf{c}, \forall \mathbf{x} \quad (15)$$

where $\mathbf{A}$ is an $LK \times LK$ matrix, $\mathbf{c}$ is a vector. If $\mathbf{A}$ is invertible, this relation is denoted as $\sim_A$.

Using the theorem, our deep latent factor model is $\sim_A$ identifiable since four conditions are satisfied:

- 1) The decoder f_d is with zero measure error.
- 2) The f_d is composed of fully connected layers and non-linear attention units, and thus not injective. Fortunately, we can assert that our attention-based structure is learned to approximate an injective according to the researches [54] and [55] on the aggregation operation of graph attention networks. According to [55], the sum aggregation in (2) is adjusted by adding 1 to α to ensure the injective.
- 3) Obviously, $\mathcal{T}$ is differentiable everywhere, and $\mathbf{h}_j^{(i)}$ and $(\mathbf{h}_j^{(i)})^2$ are linearly independent.
- 4) There exist $LK + 1$ points $\mathbf{E}_i |_{i=1}^{LK+1}$ such that the matrix $[\lambda(\mathbf{E}_2) - \lambda(\mathbf{E}_1), \dots, \lambda(\mathbf{E}_{LK+1}) - \lambda(\mathbf{E}_{LK})]$ is invertible, which is achieved by randomly and independently sampling μ and σ according to $\mathbf{E}$.

Overall, with the introduction of temporal structure as advocated in [40], we demonstrate that our model possesses identifiability. Similar to non-linear ICA models [41], [56], [57], identifiability implies that the model can uniquely infer its parameters or representations from the observed data. In other words, the learned model contains sufficient information to differentiate different factors or features, thus having the capability to generate disentangled representations. Furthermore, the disentanglement of latent factors leads to sparse connections in our model, consequently reducing the model's complexity and facilitating more efficient learning.

V. EXPERIMENTS

A. Settings

1) **Datasets:** The proposed method¹ is evaluated on two typical time series forecasting tasks, i.e., long-term series forecasting and stock trend forecasting.

For long-term series forecasting, we conduct experiments on 6 public datasets: (1) The ETTm₂ [34] dataset records a 7-dimensional feature including oil temperature and loads of an electricity transformer every 15 minutes. (2) The Exchange [31] dataset includes the daily exchange rates of 8 countries. (3) The ECL dataset collects hourly electricity consumption from 321 customers. (4) The Traffic dataset includes hourly road occupancy rates collected from 862 sensors. (5) The Weather dataset reports 21 meteorological indicators (i.e., air temperature, humidity, etc.) every 10 minutes from a weather station. (6) The ILI records weekly ratios of influenza-like illness patients. We follow standard protocols, chronologically splitting ETTm₂ into training, validation, and test sets by the ratio of 7 : 1 : 2 and other datasets by the ratio of 6 : 2 : 2. The horizon lengths are set to $H \in \{96, 192, 336, 720\}$ for the first 5 datasets, and $H \in \{24, 36, 48, 60\}$ for ILI. For the evaluation, we use the mean absolute error (MAE) and the mean squared error (MSE) metrics.

For stock trend forecasting, we conduct experiments on CSI100 and CSI300 in the publicly available stock dataset, Alpha360 [60]. The input data of this task are the sequences of 6-dimensional stock prices in 60 trading days and the output data are the price returns of the next day. For fair comparisons, we follow the experimental settings of [7]. We split the data from both sets according to temporal order, where the training, validation, and test data are sampled in the range of 01/01/2007 to 12/31/2014, 01/01/2015 to 12/31/2016, and 01/01/2017 to 12/31/2020, respectively. As for the evaluation metrics, the popular Information Coefficient (IC), Rank IC, and Precision@N (P@N) with $N \in \{3, 5, 10, 30\}$ for quantitative investment are employed. We calculate the mean and standard deviation of the results by repeating each experiment 10 times.

2) **Implementation Details:** For encoders with different sampling rates, we use 1D convolutional operations with different dilation rates to capture local contexts in temporal sequences. After comparing the validation results, we get the optimal setting, that is, factorizing the original time series into 2 or 3 signal components with different sampling rates corresponding to specific datasets, further analyzed in section V-C. The convolution kernel size is 3. We pad $(k - 1) \times r$ -dimensional zero vectors before the very first of the input features to maintain length. Residual connections via linear projections are applied to the convolution layers for better optimization. The hidden size of each latent component is set to 128. The number of the hidden units is empirically set to be 128 according to the prior related work. The coefficients of losses are selected from $\{1, 1e1, 1, 5e - 1, 1e - 1\}$ according to the corresponding performances, and the final coefficient for KL loss is determined to be $5e - 1$, while others are set to 1 for stock trend forecasting.

For long-term series forecasting, we follow the auto-tuning strategy of [17]. The learning rate is sampled between $[1e - 4, 5e - 4]$, and the input size is sampled from $\{k \times H | 1 \leq k \leq 10\}$. We emphasize the prediction loss for long-term series forecasting to enable the model to concentrate more on generating relatively hard-to-predict long-term data by multiplying the losses with $1e - 3$ except for the prediction loss. Actually, from our experiments, the longer H the better, but for computational

¹https://github.com/houjingyi-ustb/discover_PLF

TABLE I
 COMPARISONS ON THE LONG-TERM SERIES FORECASTING PERFORMANCES OF THE EXISTING AND THE PROPOSED METHODS

	H	Ours		PatchTST [58]		DLinear [13]		N-HITS [17]		NS Transformer [35]		FEDformer [36]		LSTNet [31]		DiRRNN [52]		ARIMA [59]	
		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTM ₂	96	0.135	0.203	0.166	0.256	0.167	0.260	0.176	<u>0.255</u>	0.192	0.274	0.203	0.287	3.142	1.365	0.343	0.401	0.225	0.301
	192	0.213	0.271	<u>0.223</u>	<u>0.296</u>	0.224	0.303	0.245	0.305	0.280	0.339	0.269	0.328	3.154	1.369	0.424	0.468	0.298	0.345
	336	0.257	0.324	<u>0.274</u>	<u>0.329</u>	0.281	0.342	0.295	0.346	0.334	0.361	0.325	0.366	3.160	1.369	0.632	1.083	0.370	0.386
	720	0.357	0.385	<u>0.361</u>	0.385	0.397	0.421	0.401	<u>0.413</u>	0.417	<u>0.413</u>	0.421	0.415	3.171	1.368	0.634	0.594	0.478	0.445
ECL	96	0.127	0.219	<u>0.129</u>	<u>0.222</u>	0.140	0.237	0.147	0.249	0.169	0.273	0.183	0.297	0.680	0.645	0.233	0.927	1.220	0.814
	192	0.145	<u>0.241</u>	0.147	<u>0.240</u>	0.153	0.249	0.167	0.269	0.182	0.286	0.195	0.308	0.725	0.676	0.265	0.921	1.264	0.842
	336	0.160	0.255	<u>0.163</u>	<u>0.259</u>	0.169	0.267	0.186	0.290	0.200	0.304	0.212	0.313	0.828	0.727	0.235	0.896	1.311	0.866
	720	0.197	0.287	0.197	<u>0.290</u>	<u>0.203</u>	0.301	0.243	0.340	0.222	0.321	0.231	0.343	0.957	0.811	0.322	0.890	1.364	0.891
Exchange	96	0.065	0.162	-	-	<u>0.081</u>	0.203	0.092	<u>0.202</u>	0.111	0.237	0.139	0.276	1.551	1.058	0.383	0.450	0.296	0.214
	192	0.126	0.257	-	-	<u>0.157</u>	<u>0.293</u>	0.208	0.322	0.219	0.335	0.256	0.369	1.477	1.028	1.123	0.834	1.056	0.326
	336	0.181	0.335	-	-	0.305	0.414	<u>0.301</u>	<u>0.403</u>	0.421	0.476	0.426	0.464	1.507	1.031	1.612	1.051	2.298	0.467
	720	0.476	0.515	-	-	<u>0.643</u>	0.601	0.798	<u>0.596</u>	1.092	0.769	1.090	0.800	2.285	1.243	1.827	1.131	20.666	0.864
Traffic	96	0.356	0.239	<u>0.360</u>	<u>0.249</u>	0.410	0.282	0.402	0.282	0.612	0.338	0.562	0.349	1.107	0.685	0.580	0.308	1.997	0.924
	192	0.375	0.250	<u>0.379</u>	<u>0.256</u>	0.423	0.287	0.420	0.297	0.613	0.340	0.562	0.346	1.157	0.706	0.739	0.383	2.044	0.944
	336	0.390	<u>0.266</u>	<u>0.392</u>	<u>0.264</u>	0.436	0.296	0.448	0.313	0.618	0.328	0.570	0.323	1.216	0.730	0.804	0.419	2.096	0.960
	720	0.427	0.283	<u>0.432</u>	<u>0.286</u>	0.466	0.315	0.539	0.353	0.653	0.355	0.596	0.368	1.481	0.805	0.695	0.372	2.138	0.971
Weather	96	0.145	0.187	<u>0.149</u>	0.198	0.176	0.237	0.158	<u>0.195</u>	0.173	0.223	0.217	0.296	0.594	0.587	0.193	0.245	0.217	0.258
	192	0.194	0.240	<u>0.194</u>	<u>0.241</u>	0.220	0.282	<u>0.211</u>	<u>0.247</u>	0.245	0.285	0.276	0.336	0.560	0.565	0.255	0.306	0.263	0.299
	336	<u>0.246</u>	0.278	0.245	<u>0.282</u>	0.265	0.319	0.274	0.300	0.321	0.338	0.339	0.380	0.597	0.587	0.329	0.360	0.330	0.347
	720	0.313	0.330	<u>0.314</u>	<u>0.334</u>	0.323	0.362	0.351	0.353	0.414	0.410	0.403	0.428	0.618	0.599	0.521	0.495	0.425	0.405
ILI	24	0.946	0.559	<u>1.319</u>	<u>0.754</u>	2.215	1.081	1.862	0.869	2.294	0.945	2.203	0.963	6.026	1.770	4.538	1.449	5.554	1.434
	36	0.997	0.636	<u>1.579</u>	0.870	1.963	0.963	2.071	0.934	1.825	<u>0.848</u>	2.272	0.976	5.340	1.668	3.709	1.273	6.940	1.676
	48	1.107	0.734	<u>1.553</u>	<u>0.815</u>	2.130	1.024	2.134	0.932	2.010	0.900	2.209	0.981	6.080	1.787	3.436	1.238	7.192	1.736
	60	1.260	0.753	<u>1.470</u>	<u>0.788</u>	2.368	1.096	2.137	0.968	2.178	0.963	2.545	1.061	5.548	1.720	3.703	1.272	6.648	1.656

Values marked in bold and with underlines represent the best and the second-best scores on the corresponding datasets, respectively.

sufficiency, the experimental results reported in this article are done by limiting $H < 2000$. The sampling rates (dilation rates) of different datasets contain 1, and others are chosen from basic time units, such as hour, day, week, and year. The sequence model is selected from {LSTMs, GRUs} for different tasks. The task-specific decoder is a 2-layer MLP, where the top layer is for the prediction generation and the bottom layer has 128-dimensional nodes with the ReLU activation. Because different variables at the same time step often represent homogeneous semantics in long-term forecasting datasets, we directly conduct univariate prediction following [13], [17], [58]. Actually, our method is more suitable for multivariate inputs, so we embed each 1-dimensional time series into {8, 16, 32}-dimensional sequential data using the patching operation following [58] but setting the stride to 1. To enable stable optimization, we perform data normalization within each mini-batch. Specifically, before feeding data into the proposed network, we calculate the mean and variance of all data in the mini-batch and normalize using these statistics. After generating predictions via the model, we reverse the normalization of predicted data by multiplying by the variance and adding the mean calculated initially to obtain the final prediction.

For stock trend forecasting, the learning rate is $2e - 4$. We simply apply a linear regression model to our method for end-to-end training (Ours+LR). We also cascade the knowledge-driven model, HIST [7], into our model to further improve the performance with the guidance of extra information from the market (Ours+HIST). In the experiments of this task, we further improve the model performance by applying the disentanglement trick. (7) can further be decomposed into a reconstruction term and a KL divergence between prior and posterior distributions, and the importance of the KL divergence can be gradually improved to disentangle the latent factors by tuning hyper-parameters during

the training procedure of our model. Here, the loss of the KL divergence is multiplied by a coefficient β in addition to $5e - 1$ according to the β -VAE [61] for disentangling the predictive latent factors. We gradually increase the value of β in stages, which is

$$\beta = \begin{cases} 0.1, & \text{epoch} < 20, \\ 0.5, & 20 \leq \text{epoch} < 30, \\ 1, & \text{epoch} \geq 30. \end{cases} \quad (16)$$

We observe that the optimization converges within 50 epochs.

B. Main Results

Long-term series forecasting: Table I presents the comparison of the proposed approach with related methods on 6 long-term series forecasting datasets. Our method achieves the best or the second-best performance among existing methods across all datasets. Notably, our method substantially outperforms the state-of-the-art on ETTm₂, Exchange, and ILI, which are relatively small-scale datasets containing fewer than one million observation data points and a small number of variables per time step (typically 7 or 8). Our method also outperforms the method of repeating the last values in the look-back windows according to the results as demonstrated in [13], which indicates that the mechanism of our method is unprecedented among all the existing financial models for the exchange rate prediction. We show that our method can actually capture information from the long-term observations rather than simply repeating or “guessing” the prediction from the latest observations in Section V-E. It validates that our method can handle more realistic scenarios, even with limited available information. When going deeper into the results, we can find that the MAE scores are not always better than other methods on several datasets, where most MAE scores are lower than 0.5. There are plausible

TABLE II
TEST RESULTS (AND THE CORRESPONDING STANDARD DEVIATIONS) OF THE EXISTING AND THE PROPOSED METHODS ON CSI100 AND CSI300

Methods	CSI100						CSI300					
	IC	Rank IC	P@3	P@5	P@10	P@30	IC	Rank IC	P@3	P@5	P@10	P@30
MLP	0.071 (4.8e-3)	0.067 (5.2e-3)	56.53 (0.91)	56.17 (0.48)	55.49 (0.30)	53.55 (0.36)	0.082 (6e-4)	0.079 (3e-4)	57.21 (0.39)	57.10 (0.33)	56.75 (0.34)	55.56 (0.14)
LSTM [62]	0.097 (2.2e-3)	0.091 (2.0e-3)	60.12 (0.52)	59.49 (0.19)	59.04 (0.15)	54.77 (0.11)	0.104 (1.5e-3)	0.098 (1.6e-3)	59.51 (0.46)	59.27 (0.34)	58.40 (0.30)	56.98 (0.11)
GRU [63]	0.103 (1.7e-3)	0.097 (1.6e-3)	59.97 (0.63)	58.99 (0.42)	58.37 (0.29)	55.09 (0.15)	0.113 (1.0e-3)	0.108 (8e-4)	59.95 (0.62)	59.28 (0.35)	58.59 (0.40)	57.43 (0.28)
ALSTM [64]	0.102 (1.9e-3)	0.097 (1.9e-3)	60.79 (0.23)	59.76 (0.42)	58.13 (0.13)	55.00 (0.12)	0.115 (1.4e-3)	0.109 (1.4e-3)	59.51 (0.20)	59.33 (0.51)	58.92 (0.29)	57.47 (0.16)
SFM [65]	0.081 (7.0e-3)	0.074 (8.0e-3)	57.79 (0.76)	56.96 (1.04)	55.92 (0.60)	53.88 (0.47)	0.102 (3.0e-3)	0.096 (2.7e-3)	59.84 (0.91)	58.28 (0.42)	57.89 (0.45)	56.82 (0.39)
GAT [66]	0.096 (4.5e-3)	0.090 (4.4e-3)	59.17 (0.68)	58.71 (0.52)	57.48 (0.30)	54.59 (0.24)	0.111 (1.9e-3)	0.105 (1.9e-3)	60.49 (0.39)	59.96 (0.23)	59.02 (0.14)	57.41 (0.30)
Transformer [67]	0.089 (4.7e-3)	0.090 (5.1e-3)	59.62 (1.20)	59.20 (0.84)	57.94 (0.61)	54.80 (0.33)	0.106 (3.3e-3)	0.104 (2.5e-3)	60.76 (0.35)	60.06 (0.20)	59.48 (0.16)	57.71 (0.12)
TRA [68]	0.107 (2.0e-3)	0.102 (1.8e-3)	60.27 (0.43)	59.09 (0.42)	57.66 (0.33)	55.16 (0.22)	0.119 (1.9e-3)	0.112 (1.7e-3)	60.45 (0.53)	59.52 (0.58)	59.16 (0.43)	58.24 (0.32)
FactorVAE [8]	0.107 (2.1e-3)	0.102 (1.5e-3)	60.23 (0.78)	59.51 (0.45)	58.39 (0.45)	55.21 (0.76)	0.118 (2.3e-3)	0.110 (3.1e-3)	61.04 (0.48)	60.73 (0.62)	59.26 (0.17)	57.90 (0.51)
HIST [7]*	0.120 (1.7e-3)	0.115 (1.6e-3)	61.87 (0.47)	60.82 (0.43)	59.38 (0.24)	56.04 (0.19)	0.131 (2.2e-3)	0.126 (2.2e-3)	61.60 (0.59)	61.08 (0.56)	60.51 (0.40)	58.79 (0.31)
Ours+LR	<u>0.111</u> (1.8e-3)	<u>0.105</u> (1.5e-3)	60.52 (0.63)	59.60 (0.33)	58.09 (0.25)	<u>55.31</u> (0.18)	<u>0.120</u> (2.6e-3)	<u>0.113</u> (2.4e-3)	<u>62.18</u> (0.98)	<u>61.09</u> (0.76)	<u>59.71</u> (0.47)	58.10 (0.24)
Ours+HIST*	0.128 (2.2e-3)	0.122 (1.9e-3)	62.41 (0.50)	61.41 (0.53)	60.06 (0.15)	56.46 (0.14)	0.136 (3.1e-3)	0.131 (3.2e-3)	63.08 (0.58)	62.39 (0.60)	61.59 (0.28)	59.45 (0.18)

The “*” indicates using extra knowledge. Values marked in bold and with underlines represent the highest scores with and without extra information, respectively.

explanations for this phenomenon. One potential explanation is that our method excels at predicting relatively challenging samples (i.e., those with larger error scores exceeding 0.5) but slightly worse on some easily predictable samples (i.e., those with error scores below 0.5). Alternatively, it might be attributed to the fact that there are fewer easy-to-predict samples with lower error scores than there are hard-to-predict samples with higher error scores. Achieving more precise predictions on the easy-to-predict samples is required to offset the impact of relatively inaccurate predictions on the hard-to-predict ones. Our method’s performance appears to align more with the former reason, as evidenced by its superior performance on ILI, which is rich in hard-to-predict samples, and its less ideal performance on relatively easy-to-predict datasets, i.e., ECL and Weather.

Stock trend forecasting. Table II compares the proposed method to commonly used methods for stock trend forecasting on CSI100 and CSI300. As shown, our method outperforms the state-of-the-art on all benchmarks. When combined with HIST, which exploits explicit and implicit enterprise information, our method (Ours+HIST) performs even better, suggesting that our method may be able to discover more latent factors for the time series or model the relations of the factors more efficiently. Without using extra information, our method (Ours+LR) also shows competitive results. When compared with SFM which captures multi-frequency trading patterns inside the RNN cells, our method consistently outperforms it, highlighting the advantage of processing data within the latent concept domain. Our method performs better than FactorVAE on most of the evaluation metrics, verifying its ability to discover more useful information from temporal variations.

C. Ablation Studies

We conduct ablation experiments to study the effectiveness of different modules in our method. The main modules of the proposed method are as follows:

Decomposition (Decomp.) represents that the input signal is decomposed into multiple signal components. The model without the decomposition denotes the input signal is just mapped into a single signal component.

Disentanglement (Disent.) represents the mechanism of using the KL divergence term for disentangling the inferred factors. The model without disentanglement is achieved by removing the KL divergence term from the objective function (7) during training.

Reconstruction (Recons.) represents the objective of forcing the inferred factors to reconstruct the input signals at the next time step. The model without the reconstruction is achieved by removing the reconstruction error from (7) during training.

Independence (Ind.) represents that the factors in a signal component only depend on the previous factors in the same signal component. The model without independence is implemented by concatenating all the latent factors in different signal components to predict the next factors in a signal component.

Shared Weights (S.W.) represents that the weights are shared between the combinations of the sub-decoders and the predictors.

Table III provides the experimental results of our ablation study conducted on CSI100, CSI300, Exchange, Weather, and ILI. For the 3 long-term series forecasting datasets, we report the average MAE and MSE scores of the 4 prediction horizons.

TABLE III
TEST RESULTS OF ABLATION STUDIES

Decomp.	Disent.	Reconst.	Ind.	S.W.	CSI100			CSI300			Exchange		Weather		ILI	
					IC	Rank IC	MP	IC	Rank IC	MP	MSE	MAE	MSE	MAE	MSE	MAE
×	×	×	×	×	0.103	0.098	58.15	0.113	0.108	58.92	0.289	0.397	0.251	0.310	2.089	0.977
×	✓	✓	×	×	0.105	0.100	58.26	0.116	0.110	58.96	0.285	0.366	0.246	0.303	1.661	0.808
✓	×	✓	✓	✓	0.108	0.103	58.23	0.119	0.111	60.23	0.213	0.318	0.232	0.271	1.236	0.692
✓	×	×	✓	×	0.106	0.102	58.42	0.117	0.111	60.03	0.213	0.319	0.242	0.274	1.456	0.697
✓	✓	✓	×	×	0.108	0.103	58.34	0.118	0.112	59.46	0.214	0.319	0.230	0.262	1.475	0.806
✓	✓	✓	✓	×	0.110	0.104	58.20	0.120	0.113	60.14	0.214	0.316	0.227	0.259	1.273	0.704
✓	✓	✓	✓	✓	0.111	0.105	58.38	0.120	0.113	60.27	0.212	0.317	0.225	0.259	1.078	0.671

The MP indicates the mean precision of precision@N, where $N \in \{3, 5, 10, 30\}$. The ✓ and × represent conducting experiments with and without the corresponding module, respectively.

TABLE IV
TEST RESULTS OF DIFFERENT SETTINGS OF SIGNAL COMPONENTS

Sets of sampling rates	{1}	{1,2}	{1,3}	{1,5}	{1,2,3}	{1,2,5}	{1,3,5}	{1,2,3,5}	{1,1,1}
IC	0.105	0.109	0.109	0.111	0.109	0.111	0.112	0.111	0.110
Rank IC	0.100	0.103	0.103	0.104	0.102	0.105	0.105	0.106	0.104
MP	58.26	58.32	58.27	58.41	58.32	58.38	58.34	58.36	58.31

The underlined set is selected for our method.

Results in the 1-st and 2-nd rows reveal the importance of the decomposition module in enhancing task performance. Without this model, the method achieves minimal improvement, indicating that the assumptions of predictability and independence play a crucial role in computational efficiency. From the 3-rd row of the results, we can observe that disentanglement learning is helpful for improving performance. It is probably because the semantics of the latent factors could be clarified for better representation with the disentangling operation. Moreover, we also tried other disentanglement learning methods for handling the information bottleneck problem but got no improvement. It might be because the reconstruction module helps to constrain the representation to be more informative, as supported by the findings in the 4-th row of the results. Without the guidance of the reconstruction module, learning performance tends to deteriorate. The results of the 5-th row show that the performance would be slightly lower without sharing the attention weights because sharing weights provides guidance for mapping the reconstructions and predictions into the same space, leading to better convergence. From the 6-th row of the results, we can safely conclude that there is no need to consider other signal components for the prediction of each signal component.

We investigate the settings of the signal components. We use different subsets $s \subseteq \{1, 2, 3, 5\}$ of sampling rates for investigating the settings of the signal components on CSI100. Table IV shows the comparison results. We observe that models with sampling rate 5 have relatively good performance, because they explicitly present weekly signals (5 trading days). Models with sampling rate 2 perform slightly better than those with sampling rate 3, and it might be because they can present monthly signals (20 trading days). Factorizing signal components only using sampling rate 1 can theoretically present all the signals with different periodic trends, however, the performance degradation shows that explicit settings can improve the efficiency of the

TABLE V
OPTIMAL SAMPLING RATES OF DIFFERENT DATASETS

Datasets	Frequency	Sampling rates
ETTm ₂	15min	{1,4,96}
Exchange	1day	{1,7,30}
ECL	1h	{1,168}
Traffic	1h	{1,168}
Weather	10min	{1,144}
ILI	1week	{1,2,4}

optimization. The results indicate that the model would perform better with more diverse sampling rates, but we finally choose the setting $\{1, 2, 5\}$ for computational efficiency. For the long-term series forecasting, we directly show the optimal sampling rates in Table V. Recall that the sampling rates are chosen from 1 and basic time units (i.e., hour, day, week, and month). The sampling rate of 1 can be regarded as a residual operation. For ETTm₂, Exchange and ILI, all the time units are meaningful. Too large sampling rates are not chosen, because the sampled signals are too short to be modeled. The hourly sampling is not used on the Weather dataset, which is rational because the changes of weather in the first minute and last minute in each hour do not contain any salient features. Daily variants of different hours in ECL and Traffic are redundant for prediction according to the experiments. It indicates that the daily scale contains no extra information than the hourly and weekly scales, and even introduces noise for the computation of the model.

We also conduct experiments of using different disentanglement operations for our model on CSI100, and the results are shown in Table VI. The hyper-parameters of the β -VAE_B [69] are set as follows. The starting and final annealed capacities are 0 and 0.5, respectively. The hyper-parameter, i.e., the coefficient of the TC term of the Factor-VAE [70] is set to 0.6. Note that the Factor-VAE [70] here is a totally different method from

TABLE VI
TEST RESULTS OF DIFFERENT DISENTANGLEMENT OPERATIONS

Disentanglement operations	β -VAE ($\beta=0.1$) [61]	β -VAE ($\beta=1$) [61]	β -VAE (increasing β) [61]	β -VAE _B [69]	Factor-VAE [70]
IC	0.107	0.110	0.111	0.110	0.112
Rank IC	0.102	0.104	0.105	0.105	0.106
MP	58.43	58.32	58.38	58.28	58.39

The underlined operation is selected for our method.

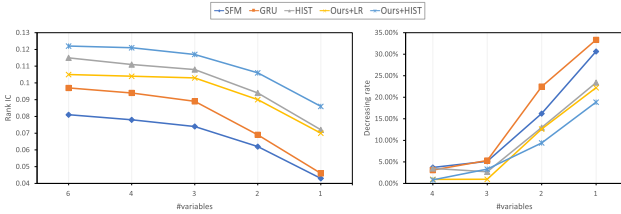


Fig. 2. Performance degradation of different methods with insufficient input data on CSI100. The left panel shows the average rank IC scores of different methods with varying numbers of variables as input. The right panel shows the decreasing rates of these scores.

the FactorVAE [8]. Although the β -VAE has the problem of information bottleneck when disentangling, and the β -VAE_B and the Factor-VAE aim to solve the problem, our method is still effective just using a simple disentanglement operation, i.e., increasing the value of β . It might be because the sufficiency of our method can alleviate the bottleneck problem. Although Factor-VAE yields the best performance, we still choose the simple strategy of increasing the β for lower time complexity.

D. Robustness to Insufficient Data

We investigate the efficacy of our method under conditions of insufficient observation data. The CSI100 dataset for stock trend prediction is employed in this experiment, where the 6 variables of the input series signify distinct attributes: “opening price”, “closing price”, “highest price”, “lowest price”, “volume weighted average price (VWAP)” and “trading volume”. We conduct experiments by gradually removing variables from the input series. The average rank IC score is computed upon removing certain numbers of different variables, along with the corresponding decreasing rate in rank IC. Note that the variables of “closing price” and “VWAP” are excluded from our experiments, as the “closing price” can directly determine the prediction, and the VWAP annotations are extremely incomplete, with most values being 0. As a result, we work with a reduced set of 4 candidate variables. Fig. 2 presents the experimental results compared with other related methods. The figure illustrates that our method declines slower than other methods, which is attributed to our method’s capacity to fully discover latent factors from historical data. While HIST also performs well, benefiting from the additional data, the result of Ours+LR affirms the heightened effectiveness of our mechanism in conditions of data insufficiency.

E. Long-Term Computational Efficiency

By leveraging the inference of predictable latent factors from observations and modeling sparse relations among them, our

method excels at capturing long-term information from observed signals. To study the long-term computational efficiency of our method, we conduct a comparative analysis with several related techniques by evaluating the forecasting performances with progressively increasing look-back window size, L , as shown in Fig. 3. Notably, when $L = 96$ on ETTm₂, the sampling rate 96 is excluded as it makes no sense for an RNN with only 1 time step input. In addition to benchmarking against 2 state-of-the-art methods, namely N-HITS and DLinear (both implemented using the authors’ provided source codes), we evaluated 3 additional methods to explore specific mechanisms for handling long-term series of our method. These include our backbone model - a fusion of 1D convolutional layers and LSTMs (ConvLSTM), cascaded 1D dilated convolutional layers employing the same dilation rates as our method (TCN), and our model without the independence operation (ours wo Ind.).

The figure shows that our methods can constantly extract useful information from observations. This is evident from the generally ascending trend in our method’s performances because our method’s performance exhibits a generally ascending trend with the look-back window size increasing on both datasets. For the Exchange dataset, the lowest MSE scores are found between the look-back window sizes of 720 and 960. This trend likely arises from the model’s limited capability to process extensive information on the Exchange dataset. However, it is more likely that longer series no longer contain additional useful information for forecasting, as evidenced by the majority of optimal results of other methods being observed at $L \leq 336$. The trend pattern of ours wo Ind. is similar to our method, suggesting the overall framework of our method facilitates the extraction of long-term information, and the independent operation further improves the efficacy. ConvLSTM shows its effectiveness in long-term computation, primarily owing to the long-term memory mechanism of LSTM. We believe that our method also benefits from this aspect. Conversely, cascaded multi-scale sampling without decomposition contributes minimally to long-term computational efficiency improvement, as indicated by TCN’s outcomes. In conclusion, the long-term computational efficiency of our method stems primarily from the independence operation, the long-term memory mechanism, and the parallel multi-scale sampling with decomposition.

F. Statistical Analysis

The predictability of the latent factor is encouraged in our model. According to (8), stationarity is a necessary condition of a predictable time series, since stationarity ensures that the function g is able to output similar values for similar inputs. It’s difficult to design appropriate statistics for testing predictability,

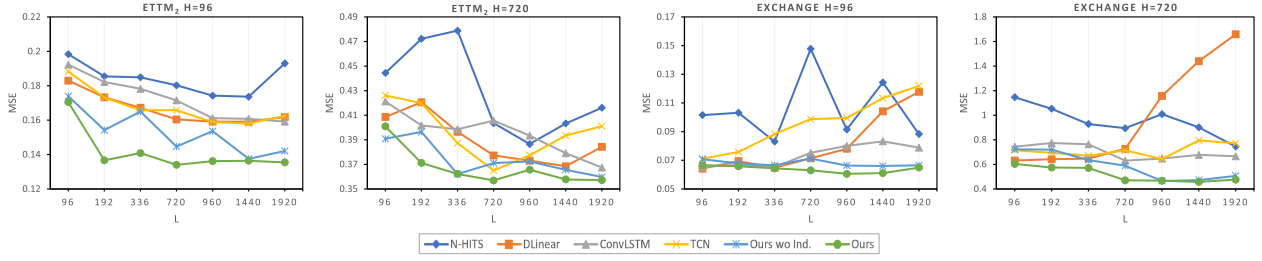


Fig. 3. Test results (MSE) of prediction horizons $H = 96, 720$ with varying look-back windows; $L = 96, 192, 336, 720, 960, 1440, 1920$ on ETTm₂ and Exchange.

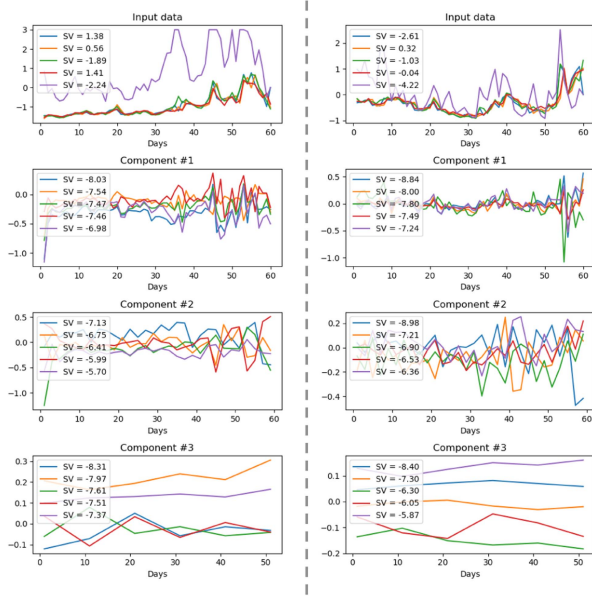


Fig. 4. Examples of the ADF tests on the original signal and learned factors. Each column presents the visualization results of a randomly selected sample.

but lots of hypothesis test methods for stationarity are proposed. Here, the classical Augmented Dickey-Fuller unit root test (ADF test) is used on both the input original price signal and the learned latent factors, and Fig. 4 depicts the test results of 2 randomly selected samples. The first row shows the original price paths along time (we neglect the VWAP, because the values are almost zero). The other rows show the signal components with each color representing a representative latent factor with the topmost statistic values (SVs). As expected, the SVs of the learned factors are commonly less than the critical value under the significance level of 1%, 5% or 10%, i.e., $-3.46, -2.87, -2.57$, indicating that the latent factor series is stationary. The ADF tests the linear reliability contained in the time series, and it is interesting to note that our factors can still refuse the null hypothesis even though the g in the proposed method is implemented via GRU which is a non-linear neural network. The probable reason is that the learned g actually approximates an injective to fit the complex data for predictability, which is more strict than the stationarity.

To study the interpretability of the inferred latent factors, we draw heatmaps illustrating the correspondence between latent factors and predefined concepts within the stock market, as

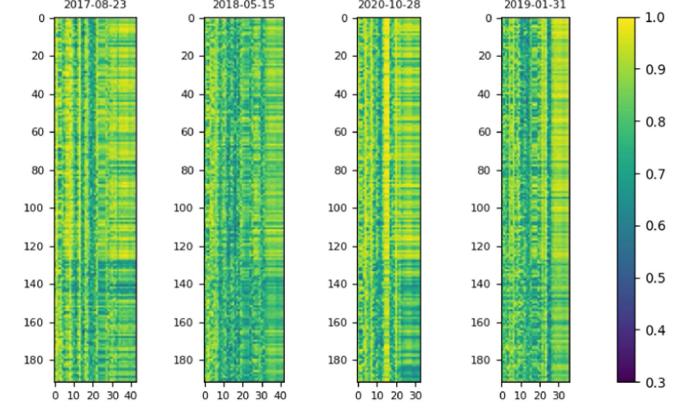


Fig. 5. Visualization of correspondences between latent factors and predefined concepts in the stock market on CSI100 on four different days. The x-axis represents concept indices and the y-axis lists latent factor indices. The correspondence is measured by the ROC-AUC score calculated by classifying the stocks into a certain concept according to the clustering results of the time series of a latent factor. A higher correspondence value indicates that the latent factor is more likely to share some meaningful properties with an explicit concept.

depicted in Fig. 5. This exploration aims to determine whether the latent factors bear relevance to tangible real-world semantics. Thankfully, the Alpha360 dataset provides annotations to predefined concepts in the stock market, such as technology and E-Commerce, and these annotations are reprocessed as a binary matrix with each element assuming a value of 0 or 1, denoting whether a stock belongs to be a given concept by [7], and used in their HIST framework as extra knowledge. We postulate that stocks categorized under the same predefined stock concept share analogous inherent patterns. Some of these patterns may align with our learned latent factors. If a simple unsupervised classifier can accurately recognize whether the stocks belong to a concept with one of the latent factors, it implies that the latent factor represents meaningful characteristics related to the concept. If the concept classification results derived from a latent factor are constantly the same over time, it would further validate that our model can learn the latent concept with a particular semantic. Accordingly, we use k -means clustering to classify the stocks with the 60-dimensional historical data of each latent factor as features. For each concept and each latent factor, we compute the ROC-AUC scores for all the clusters. The highest of these scores is selected as a quantification of the correspondence between the given concept and the latent factor.

In this experiment, we set the number of latent factors in each signal component as 64 for clear visualization. Note that we remove concepts that contain less than 3 stocks, and because the stocks vary over time, the concepts of each day would be slightly different. And the order of the concepts remains unchanged. From the figure, many latent factors show high correspondence between the stock concepts, and the whole pattern of each day is generally the same, demonstrating that the latent factors are very likely to have meaningful semantics.

VI. CONCLUSION

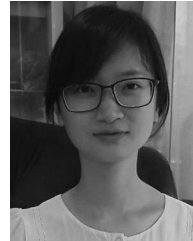
We have introduced a simple but efficient method of inferring predictable latent factors for time series forecasting. The inferred factors can form multiple independent signal components with characteristics of predictability, sufficiency, and identifiability to guarantee that our model effectively reconstructs the future values with these components in the latent concept domain. Thanks to the independence and predictability of the discovered signal components, our method enables sparse relation modeling of the inferred factors for long-term efficiency and the easy reconstruction of the prediction. Experiments validate the effectiveness of the proposed method and the predictability of the inferred latent factors.

The limitation of our work is that we empirically sample data of different time scales for inferring the latent factors and the signal components with specific inductive bias. In the future, we plan to explore a more general method that enables the model to learn the optimal encoding structure by adding a meta-learning algorithm.

REFERENCES

- [1] C. Feichtenhofer, H. Fan, J. Malik, and K. He, "Slowfast networks for video recognition," in *Proc. Int. Conf. Comput. Vis.*, 2019, pp. 6201–6210.
- [2] J. Cheng, L. Dong, and M. Lapata, "Long short-term memory-networks for machine reading," in *Proc. Conf. Emp. Methods Natural Lang. Process.*, 2016, pp. 551–561.
- [3] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [4] Y. Xu and S. B. Cohen, "Stock movement prediction from tweets and historical prices," in *Proc. Assoc. For Comput. Linguist.*, 2018, pp. 1970–1979.
- [5] L. Shi, Z. Teng, L. Wang, Y. Zhang, and A. Binder, "DeepClue: Visual interpretation of text-based deep stock prediction," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 6, pp. 1094–1108, Jun. 2019.
- [6] D. Cheng, F. Yang, S. Xiang, and J. Liu, "Financial time series forecasting with multi-modality graph neural network," *Pattern Recognit.*, vol. 121, 2022, Art. no. 108218.
- [7] W. Wu et al., "HIST: A graph-based framework for stock trend forecasting via mining concept-oriented shared information," 2022, *arXiv:2110.13716*.
- [8] Y. Duan, L. Wang, Q. Zhang, and J. Li, "Factorvae: A probabilistic dynamic factor model based on variational autoencoder for predicting cross-sectional stock returns," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 4468–4476.
- [9] R. Asadi and A. C. Regan, "A spatio-temporal decomposition based deep neural network for time series forecasting," *Appl. Soft Comput.*, vol. 87, 2020, Art. no. 105963.
- [10] B. N. Oreshkin, D. Carpio, N. Chapados, and Y. Bengio, "N-BEATS: Neural basis expansion analysis for interpretable time series forecasting," in *Proc. Int. Conf. Learn. Represent.*, 2020.
- [11] H. Wu, J. Xu, J. Wang, and M. Long, "AutoFormer: Decomposition transformers with auto-correlation for long-term series forecasting," in *Proc. Adv. Neural Inform. Process. Syst.*, 2021, pp. 22419–22430.
- [12] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning, "STL: A seasonal-trend decomposition," *J. Off. Stat.*, vol. 6, no. 1, pp. 3–73, 1990.
- [13] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 11121–11128.
- [14] N. E. Huang et al., "The empirical mode decomposition and the Hilbert spectrum for nonlinear and non-stationary time series analysis," *Proc. Roy. Soc. London. Ser. A: Math. Phys. Eng. Sci.*, vol. 454, no. 1971, pp. 903–995, 1998.
- [15] N. E. Huang, *Hilbert-Huang Transform and its Applications*, vol. 16. Singapore: World Scientific, 2014.
- [16] K. Dragomiretskiy and D. Zosso, "Variational mode decomposition," *IEEE Trans. Sign. Process.*, vol. 62, no. 3, pp. 531–544, Feb. 2014.
- [17] C. Challu, K. G. Olivares, B. N. Oreshkin, F. Garza, M. Mergenthaler, and A. Dubrawski, "N-HiTS: Neural hierarchical interpolation for time series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 6989–6997.
- [18] C. A. Sims, "Macroeconomics and reality," *Econ.: J. Econ. Soc.*, vol. 48, no. 1, pp. 1–48, 1980.
- [19] G. Tauchen, "Finite state markov-chain approximations to univariate and vector autoregressions," *Econ. Lett.*, vol. 20, no. 2, pp. 177–181, 1986.
- [20] G. E. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*. Hoboken, NJ, USA: Wiley, 2015.
- [21] G. W. Morrison and D. H. Pike, "Kalman filtering applied to statistical forecasting," *Manage. Sci.*, vol. 23, no. 7, pp. 768–774, 1977.
- [22] T. W. Joo and S. B. Kim, "Time series forecasting based on wavelet filtering," *Expert Syst. Appl.*, vol. 42, no. 8, pp. 3868–3874, 2015.
- [23] W. F. Sharpe, "Capital asset prices: A theory of market equilibrium under conditions of risk," *J. Finan.*, vol. 19, no. 3, pp. 425–442, 1964.
- [24] E. F. Fama and K. R. French, "The cross-section of expected stock returns," *J. Finan.*, vol. 47, no. 2, pp. 427–465, 1992.
- [25] S. S. Rangapuram, M. W. Seeger, J. Gasthaus, L. Stella, Y. Wang, and T. Januschowski, "Deep state space models for time series forecasting," in *Proc. Adv. Neural Inform. Process. Syst.*, 2018, pp. 7796–7805.
- [26] V. Flunkert, D. Salinas, and J. Gasthaus, "DeepAR: Probabilistic forecasting with autoregressive recurrent networks," *Int. J. Forecast.*, vol. 36, no. 3, pp. 1181–1191, 2020.
- [27] Y. Qin, D. Song, H. Chen, W. Cheng, G. Jiang, and G. W. Cottrell, "A dual-stage attention-based recurrent neural network for time series prediction," in *Proc. Int. Joint Conf. Artif. Intell.*, 2017, pp. 2627–2633.
- [28] H. Song, D. Rajan, J. J. Thiagarajan, and A. Spanias, "Attend and diagnose: Clinical time series analysis using attention models," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 4091–4098.
- [29] F. Feng, H. Chen, X. He, J. Ding, M. Sun, and T. Chua, "Enhancing stock movement prediction with adversarial training," in *Proc. Int. Joint Conf. Artif. Intell.*, 2019, pp. 5843–5849.
- [30] A. Borovykh, S. Bohte, and C. W. Oosterlee, "Conditional time series forecasting with convolutional neural networks," 2017, *arXiv: 1703.04691*.
- [31] G. Lai, W. Chang, Y. Yang, and H. Liu, "Modeling long- and short-term temporal patterns with deep neural networks," in *Proc. Int. Conf. Res. Dev. Inf. Retrieval*, 2018, pp. 95–104.
- [32] R. Sen, H. Yu, and I. S. Dhillon, "Think globally, act locally: A deep neural network approach to high-dimensional time series forecasting," in *Proc. Adv. Neural Inform. Process. Syst.*, 2019, pp. 4838–4847.
- [33] N. Kitaev, L. Kaiser, and A. Levskaya, "Reformer: The efficient transformer," in *Proc. Int. Conf. Learn. Representations*, 2020.
- [34] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 11106–11115.
- [35] Y. Liu, H. Wu, J. Wang, and M. Long, "Non-stationary transformers: Exploring the stationarity in time series forecasting," in *Proc. Adv. Neural Inform. Process. Syst.*, 2022, pp. 9881–9893.
- [36] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 27268–27286.
- [37] D. Hafner et al., "Learning latent dynamics for planning from pixels," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 2555–2565.
- [38] P. Becker, H. Pandya, G. H. W. Gebhardt, C. Zhao, C. J. Taylor, and G. Neumann, "Recurrent Kalman networks: Factorized inference in high-dimensional deep feature spaces," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 544–552.
- [39] V. Saxena, J. Ba, and D. Hafner, "Clockwork variational autoencoders," in *Proc. Adv. Neural Inform. Process. Syst.*, 2021, pp. 29246–29257.

- [40] I. Khemakhem, D. P. Kingma, R. P. Monti, and A. Hyvärinen, "Variational autoencoders and nonlinear ICA: A unifying framework," in *Proc. Int. Conf. Artif. Intell. Stat.*, 2020, pp. 2207–2217.
- [41] H. Hälvä et al., "Disentangling identifiable features from noisy data with structured nonlinear ICA," in *Proc. Adv. Neural Inform. Process. Syst.*, 2021, pp. 1624–1633.
- [42] Y. Chen, Z. Wei, and X. Huang, "Incorporating corporation relationship via graph convolutional neural networks for stock price prediction," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2018, pp. 1655–1658.
- [43] R. Kim, C. H. So, M. Jeong, S. Lee, J. Kim, and J. Kang, "HATS: A hierarchical graph attention network for stock movement prediction," 2019, *arXiv: 1908.07999*.
- [44] Z. Wu, S. Pan, G. Long, J. Jiang, X. Chang, and C. Zhang, "Connecting the dots: Multivariate time series forecasting with graph neural networks," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2020, pp. 753–763.
- [45] Y. Chen, I. Segovia-Dominguez, B. Coskunuzer, and Y. R. Gel, "TAMP-S2GCNets: Coupling time-aware multipersistance knowledge representation with spatio-supra graph convolutional networks for time-series forecasting," in *Proc. Int. Conf. Learn. Represent.*, 2022.
- [46] C. Li, D. Song, and D. Tao, "Multi-task recurrent neural networks and higher-order Markov random fields for stock price movement prediction: Multi-task RNN and higher-order MRFs for stock price classification," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2019, pp. 1141–1151.
- [47] V. Ng, R. F. Engle, and M. Rothschild, "A multi-dynamic-factor model for stock returns," *J. Econometrics*, vol. 52, no. 1/2, pp. 245–266, 1992.
- [48] C. Jutten and J. Hérault, "Blind separation of sources, part I: An adaptive algorithm based on neuromimetic architecture," *Sign. Process.*, vol. 24, no. 1, pp. 1–10, 1991.
- [49] H. Wang, J. Peng, F. Huang, J. Wang, J. Chen, and Y. Xiao, "MICN: Multi-scale local and global context modeling for long-term series forecasting," in *Proc. Int. Conf. Learn. Representations*, 2023.
- [50] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," 2018, *arXiv: 1803.01271*.
- [51] Z. Yue et al., "TS2Vec: Towards universal representation of time series," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 8980–8987.
- [52] S. Chang et al., "Dilated recurrent neural networks," in *Proc. Adv. Neural Inform. Process. Syst.*, 2017, pp. 77–87.
- [53] U. A. Müller, M. M. Dacorogna, R. D. Davé, O. V. Pictet, R. B. Olsen, and J. R. Ward, "Fractals and intrinsic time: A challenge to econometricians," *Unpublished Manuscript*, Zürich, Switzerland: Olsen & Associates, 1993, Art. no. 130.
- [54] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [55] A. Wijesinghe and Q. Wang, "A new perspective on "how graph neural networks go beyond Weisfeiler-Lehman?," in *Proc. Int. Conf. Learn. Representations*, 2022.
- [56] F. Locatello et al., "Challenging common assumptions in the unsupervised learning of disentangled representations," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 4114–4124.
- [57] A. Hyvärinen and H. Morioka, "Unsupervised feature extraction by time-contrastive learning and nonlinear ICA," in *Proc. Adv. Neural Inform. Process. Syst.*, 2016, pp. 3765–3773.
- [58] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *Proc. Int. Conf. Learn. Representations*, 2023.
- [59] O. Anderson, *Time-Series*, 2nd ed., London, U.K.: Griffin, 1976.
- [60] X. Yang, W. Liu, D. Zhou, J. Bian, and T. Liu, "Qlib: An AI-oriented quantitative investment platform," 2020, *arXiv: 2009.11189*.
- [61] I. Higgins et al., "beta-VAE: Learning basic visual concepts with a constrained variational framework," in *Proc. Int. Conf. Learn. Representations*, 2017.
- [62] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [63] J. Chung, Ç. Gülçehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014, *arXiv:1412.3555*.
- [64] A. A. Ariyo, A. O. Adewumi, and C. K. Ayo, "Stock price prediction using the ARIMA model," in *Proc. UKSim-AMSS Int. Conf. Comput. Modelling Simul.*, 2014, pp. 106–112.
- [65] L. Zhang, C. C. Aggarwal, and G. Qi, "Stock price prediction via discovering multi-frequency trading patterns," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2017, pp. 2141–2149.
- [66] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018.
- [67] Q. Ding, S. Wu, H. Sun, J. Guo, and J. Guo, "Hierarchical multi-scale Gaussian transformer for stock movement prediction," in *Proc. Int. Joint Conf. Artif. Intell.*, 2020, pp. 4640–4646.
- [68] H. Lin, D. Zhou, W. Liu, and J. Bian, "Learning multiple stock trading patterns with temporal routing adaptor and optimal transport," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2021, pp. 1017–1026.
- [69] C. P. Burgess et al., "Understanding disentangling in -VAE," 2018, *arXiv: 1804.03599*.
- [70] H. Kim and A. Mnih, "Disentangling by factorising," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 2654–2663.



Jingyi Hou received the BE degree in electrical engineering and automation from the China University of Mining and Technology, Beijing, China, in 2014 and the PhD degree in computer science and technology from the Beijing Institute of Technology, Beijing, in 2020. She is currently a lecturer with the University of Science and Technology Beijing, Beijing. Her research interests include time series analysis, data mining, and video content analysis.



Zhen Dong received the BS and PhD degrees in computer science and technology from the Beijing Institute of Technology, Beijing, China, in 2011 and 2017, respectively. His main research interests are machine learning, data mining, fintech, and their applications on quantitative investment.



Jiayu Zhou (Member, IEEE) received the PhD degree in computer science from Arizona State University, in 2014. He is an associate professor with the Department of Computer Science and Engineering, Michigan State University. He has a broad research interest in large-scale machine learning and data mining, and biomedical informatics. He served as technical program committee member of premier conferences such as NIPS, ICML, and SIGKDD. His research is supported by the National Science Foundation, National Institutes of Health, and Office of Naval Research. He is a recipient of the National Science Foundation CAREER Award (2018). His papers received the Best Student Paper Award in the 2014 IEEE International Conference on Data Mining (ICDM); the Best Student Paper Award at the 2016 International Symposium on Biomedical Imaging (ISBI); Best Paper Award in the 2016 IEEE International Conference on Big Data (BigData); and Best Paper Award in Health Track, the 2022 SIGKDD Conference on Knowledge Discovery and Data Mining.



Zhijie Liu (Member, IEEE) received the BE degree from the China University of Mining and Technology, Beijing, China, in 2014, and the PhD degree from Beihang University, Beijing, in 2019. In 2017, he was a research assistant with the Department of Electrical Engineering, University of Notre Dame, Notre Dame, IN, USA, for 12 months. He is currently a professor with the School of Intelligence Science and Technology, University of Science and Technology Beijing, Beijing. His research interests include adaptive control, modeling and vibration control for flexible structures, and intelligent control systems. He is a member of the IEEE SMC Technical Committee on Autonomous Bionic Robotic Aircraft. He was a recipient of the IEEE SMC Beijing Capital Region Chapter Young Author Prize Award, in 2019.